

MODULE 9 · TROUBLESHOOTING

DNS Problems

Check CoreDNS first, then narrow the scope

Almost every "can't reach the Service" starts as DNS

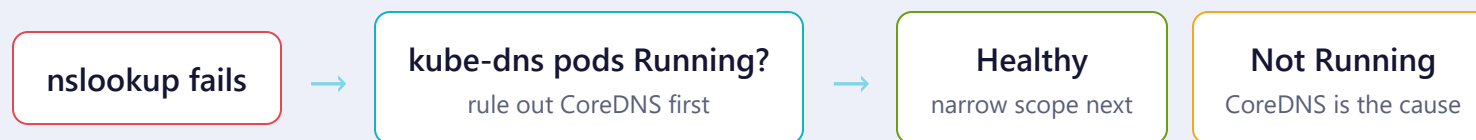
CoreDNS is what turns a Service name into an IP. It's just another Pod — it can be misconfigured, blocked, or scaled to zero like anything else.

Four failure modes cover almost everything: a **name that doesn't exist**, an **egress policy blocking port 53**, **CoreDNS itself down**, and a **dnsPolicy pitfall** on one pod.

BY THE END YOU CAN

- Rule CoreDNS in or out in one command
- Tell "one name" apart from "clusterwide"
- Fix a NetworkPolicy that ate DNS

Check the resolver before you check the name



If CoreDNS is healthy, the problem is the name, the path, or one pod's policy — not the resolver. If it isn't Running, stop there: nothing else matters until it's back.

What the failure looks like

SYMPTOM	SCOPE	LIKELY CAUSE	DIAGNOSTIC
<code>NXDOMAIN</code> for one name	Single query	Typo, or Service in a different namespace	<code>kubect1 get svc -A</code>
Timeout right after a NetworkPolicy was added	One pod	Egress default-deny blocking UDP/TCP 53	<code>get networkpolicy</code>
<code>SERVFAIL</code> /timeout, every pod, every namespace	Clusterwide	CoreDNS scaled down or crashing	<code>kube-system get pods -l k8s-app=kube-dns</code>
One pod's <code>resolv.conf</code> looks nothing like its neighbours'	Single pod	<code>dnsPolicy: Default</code> (inherits node resolver)	<code>-o jsonpath='{.spec.dnsPolicy}'</code>

Two commands, in this order

- **CoreDNS health first** — Running pods rule out the resolver as the cause
- **nslookup second** — the failure text tells you which mode you're in

ORDER MATTERS

If CoreDNS is down, every lookup times out. Don't chase a single Service typo when the whole cluster is dark.

```
$ kubectl -n kube-system get pods -l k8s-app=kube-dns
# Running x2 = CoreDNS is healthy

$ kubectl exec POD -n NS -- nslookup NAME
# NXDOMAIN = name doesn't exist
# timeout = packets never arrived
```

A default-deny takes DNS down with it

An egress NetworkPolicy selecting a pod with **no** egress rules denies all outbound traffic — including UDP/TCP 53 to CoreDNS.
`/etc/resolv.conf` is unchanged; the packets just never arrive.

The tell: a lookup that worked a moment ago now times out for **every** name, right after a policy was applied.

FIX

Add a policy selecting the same pod that explicitly allows port 53 to `k8s-app=kube-dns` — policies on the same pod are additive.

```
$ kubectl exec dns-client -- nslookup svc
;; connection timed out;
   no servers could be reached

$ kubectl describe networkpolicy deny-egress
Policy Types: Egress
   (no egress rules = deny all)
```

The most severe mode: every pod, every namespace

CoreDNS scaled to zero or crashing. The kube-dns Service's ClusterIP still exists — every pod's `resolv.conf` still points at it — but its Endpoints are empty. Queries go nowhere and time out.

Blast radius is the tell: this hits pods with no NetworkPolicy at all, in every namespace — not just one.

```
$ kubectl -n kube-system get deployment coredns
# 0/0 ready

$ kubectl -n kube-system get endpoints kube-dns
# <none>

$ kubectl -n kube-system scale deployment coredns --
replicas=2
```

Where people trip

- **Chasing the Service name** before checking CoreDNS — rule out the resolver first.
- **dnsPolicy: Default** doesn't mean "the default" — it skips CoreDNS for the node's own resolver. The real default is `ClusterFirst`.
- **Forgetting the namespace segment.** A bare short name only resolves inside its own namespace.
- **Leaving CoreDNS scaled to 0** after testing — breaks DNS clusterwide, not just your Pod.

Commands to keep at hand

```
$ kubectl -n kube-system get pods -l k8s-app=kube-dns      # CoreDNS health, first
$ kubectl exec POD -n NS -- nslookup NAME                 # NXDOMAIN vs timeout
$ kubectl exec POD -n NS -- cat /etc/resolv.conf           # resolver + search domains
$ kubectl get networkpolicy -n NS                         # egress rules blocking :53
$ kubectl -n kube-system get deployment coredns endpoints kube-dns
```

Tip: CoreDNS health, then nslookup, then narrow the scope — one pod vs. every pod.

RECAP

DNS triage in one breath

- Is CoreDNS healthy? Check that before anything else
- One name fails → typo/namespace; one pod fails everything → NetworkPolicy; every pod fails → CoreDNS itself
- `dnsPolicy`: Default skips CoreDNS entirely
- Compare `resolv.conf` across pods to spot the odd one out

HANDS-ON

Now run Lab 9.5
`labs/9.5/`

Next: [9.6 — Ingress Problems](#)